

FIELD PROGRAMME RAHIM – WEEK 7

IoT, ESP32 Simulation & Real-Time Motor Monitoring

By Friday, you should be able to:

- Understand how an ESP32-based IoT system works.
- Understand how sensors collect motor data.
- Understand the role of temperature, vibration, sound and current sensors.
- Understand how sensor data travels from a device to a computer/server.
- Understand serial communication.
- Understand HTTP APIs and basic MQTT concepts.
- Simulate an ESP32 and motor sensors without physical hardware.
- Send simulated sensor readings to a Python application.
- Receive and process sensor data in real time.
- Apply the Machine Learning model developed in Week 6 to incoming sensor data.
- Display motor health information on a simple dashboard.

Day 1 – Understanding the Hardware Architecture

This day is **mostly learning**, because I don't you jumping into ESP32 code without understanding what he's building.

Learn

Study:

What is an ESP32?

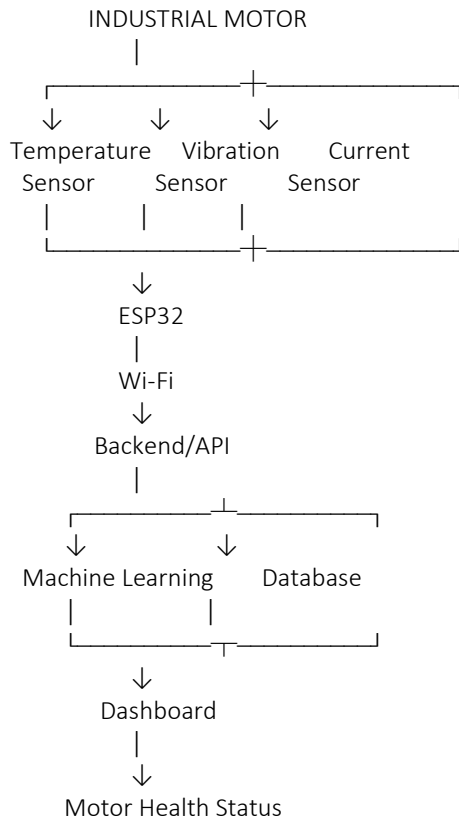
Understand:

- Microcontroller
- GPIO
- ADC
- Digital sensors
- Analog sensors
- Wi-Fi

- Bluetooth
 - Serial communication
-

Understand the Project Architecture

He should understand this:



Understand Each Sensor

He should research what kind of sensor could be used for:

Temperature

For example:

- DS18B20
- DHT22

Vibration

For example:

- MPU6050
- ADXL345

Current

For example:

- ACS712
- SCT-013

Sound

For example:

- Microphone module
- MEMS microphone

Don't worry about purchasing anything yet.

The goal is simply to understand:

What does the sensor measure, what kind of output does it provide, and how would an ESP32 read it?

Practical Task

Create:

hardware_research.md

For each sensor, document:

Sensor:

What it measures:

Type of output:

How ESP32 would read it:

Possible use in motor monitoring:

Limitations:

Architecture Diagram

Create your own architecture diagram for your FYP.

Something like:

```
graph TD; Motor --> Sensors; Sensors --> ESP32; ESP32 --> Wi-Fi; Wi-Fi --> Server; Server --> ML_Model[ML Model]; ML_Model --> Dashboard;
```

Motor
↓
Sensors
↓
ESP32
↓
Wi-Fi
↓
Server
↓
ML Model
↓
Dashboard

explain every arrow.

Day 2 – Simulating the ESP32 & Sensors

Since we don't have hardware, we will do it virtually

Goal

Create a **virtual motor + virtual ESP32**.

Instead of:

REAL SENSOR
↓
ESP32

we'll temporarily use:

PYTHON SENSOR SIMULATOR
↓
ESP32-LIKE
↓
INTERNET/API

This lets you develop most of the software before buying hardware.

Learn

Study:

- Sensor readings
- Sampling
- Sampling interval
- Serial communication
- JSON
- REST APIs
- HTTP POST
- HTTP GET

Practical Work

Create:

motor_sensor_simulator.py

It should generate readings such as:

Temperature: 62.4

Vibration: 0.18

Current: 8.4

Sound: 32

RPM: 1482

Every few seconds.

For example:

10:00:01

Temperature = 61.2

Vibration = 0.16

Current = 8.2

Sound = 31

RPM = 1480

10:00:03

Temperature = 61.5

Vibration = 0.17

Current = 8.3

Sound = 32

RPM = 1481

Simulate Motor Conditions

Create three modes:

Healthy

Normal sensor values

Warning

Slightly abnormal readings

Fault

Strongly abnormal readings

For example:

HEALTHY

Temperature → ~60

Vibration → low

Current → normal

Sound → normal

Then:

WARNING

Temperature → increasing

Vibration → increasing

Current → slightly high

And:

FAULT

Temperature → very high

Vibration → very high

Current → high

RPM → dropping

Important

Don't simply make:

```
if mode == "fault":
```

temperature = 90

and call it finished.

Make the simulated data **change gradually**.

For example:

60
61
62
64
67
71
76
82

This is much more realistic and prepares you for the time-series aspect of the FYP.

Day 3 – Send Sensor Data to a Backend

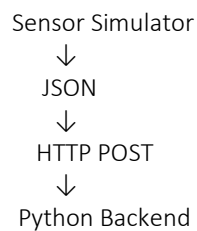
Now we connect the simulated sensor to a server.

Learn

Study:

- REST APIs
- HTTP
- POST requests
- JSON
- API endpoints
- Client/server architecture

The architecture becomes:



Practical Work

Create a simple Python API.

You can use:

FastAPI

Install:

```
pip install fastapi uvicorn
```

Create an endpoint such as:

POST /sensor-data

The simulator sends:

```
{  
  "temperature": 62.4,  
  "vibration": 0.18,  
  "current": 8.4,  
  "sound": 32,  
  "rpm": 1482  
}
```

The backend receives it.

Backend Task

When data arrives:

1. Validate it.
2. Print it.
3. Add timestamp.
4. Store it.

For now, storage can simply be:

`sensor_data.csv`

or SQLite.

Don't over-engineer this.

Deliverable

He should be able to run:

`motor_sensor_simulator.py`

and see:

Sending sensor data...

Temperature: 62.4

Vibration: 0.18

Current: 8.4

Sound: 32

RPM: 1482

Data sent successfully.

while the backend receives it.

Day 4 – Connect Machine Learning to Live Data

This is where Week 6 comes alive.

We now have:

Simulated Sensors



Backend



ML Model

Learn

Study:

- Loading a trained model
- Real-time prediction
- Prediction probability
- Classification thresholds
- Model inference

Practical Work

Take the model from Week 6.

Load:

`motor_model.pkl`

When new sensor data arrives:

Temperature: 74

Vibration: 0.42

Current: 9.8

RPM: 1390

send it to the model.

The model returns:

Prediction: WARNING

Confidence: 82%

Create Motor Health Logic

For this prototype, use:

HEALTHY
WARNING
FAULT

For example:

Model prediction
↓
Health Status
↓
Dashboard

Important Concept

Learn the distinction between:

Sensor threshold

Temperature > X

and:

Machine Learning prediction

Temperature
+
Vibration
+
Current
+
RPM
+
Sound
↓
ML Model
↓
Prediction

The second is what his FYP is actually trying to investigate.

Day 5 – Build the Real-Time Dashboard

Now we create the first **end-to-end** prototype.

You don't need a fancy frontend.

Use:

Streamlit

Install:

```
pip install streamlit
```

Dashboard

Create:

```
dashboard.py
```

The dashboard should show:

```
SMART AI POWER MONITOR
=====
```

Motor Status

 HEALTHY

Temperature
62.4 °C

Current
8.4 A

Vibration
0.18

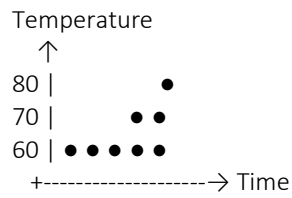
Sound
32 dB

RPM
1482

Add Real-Time Graphs

Display:

Temperature



Vibration

Current

RPM

Add Health History

Show something like:

Time	Temperature	Vibration	Status
10:00:01	60.2	0.15	HEALTHY
10:00:05	61.1	0.16	HEALTHY
10:00:10	64.3	0.19	HEALTHY
10:00:15	69.8	0.25	WARNING
10:00:20	75.2	0.38	WARNING
10:00:25	82.1	0.52	FAULT



Add an Alert

When the model predicts a fault:

⚠️ MOTOR FAULT DETECTED

The motor may require inspection.

Temperature: 82.1°C

Vibration: 0.52

Current: 10.8A

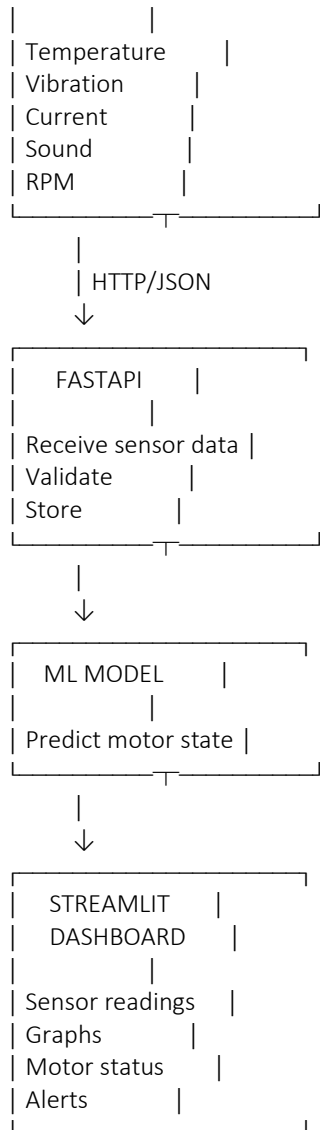
RPM: 1290

Again, make sure you understands that this is a **prototype prediction**, not a certified safety system.

End-to-End Architecture

By Friday, he should have:

┌
| MOTOR SIMULATOR |
└



Friday Demonstration

Start the system with the motor in HEALTHY mode.

Dashboard:

 HEALTHY

Then change the simulator to:

WARNING

The readings should gradually change.

Dashboard:

 WARNING

Then switch to:

FAULT

The sensor values should deteriorate.

Dashboard:

 FAULT

And an alert should appear.

Friday Questions

Hardware

What does the ESP32 do?

Sensors

How would you measure vibration?

Communication

How does sensor data get from the ESP32 to our server?

Backend

Why are we using an API?

Machine Learning

What happens when new sensor data arrives?

Dashboard

How does the dashboard know that the motor's condition changed?

Real World

Then ask:

What would we need to change if we replaced your Python simulator with a real ESP32?

The ideal answer is something like:

Python Simulator
↓
REMOVE
↓
ESP32
↓
Real Sensors
↓
Wi-Fi
↓
Same Backend
↓
Same ML Model
↓
Same Dashboard

And **that is exactly the seamless transition you're looking for.**

Week 7 Deliverables

Your repository should roughly become:

```
python-learning/  
|
```

```
├── week1/
├── week2/
├── week3/
├── week4/
├── week5/
├── week6/
└── week7/
    ├── hardware_research.md
    ├── simulator/
    │   └── motor_sensor_simulator.py
    ├── backend/
    │   └── main.py
    ├── ml/
    │   └── motor_model.pkl
    ├── dashboard/
    │   └── dashboard.py
    ├── data/
    │   └── sensor_data.csv
    └── README.md
```